Written Report

Carlos Granillo

Date:1/28/2026

Lab02 – ITAI 3377

During this assignment, I built, trained, and deployed a simple neural network using the MNIST handwritten digit dataset. The goal was to understand the full machine learning process, from preparing the data to running predictions on a small, real-world-friendly model.

The first step was loading and normalizing the MNIST dataset so that all pixel values were scaled between 0 and 1. Normalization helps the model learn more efficiently and I also displayed some sample images to see what the model would be learning each image showed a handwritten number from 0 to 9. After this, I created a neural network using TensorFlow and Keras. The model had three total layers, so I trained the model and checked its accuracy. After training it, the model was saved and converted into TensorFlow Lite format, making it smaller and faster to run on devices.

One of the challenges that I faced was dealing with data shapes and formatting. Even small mistakes, like mismatched tensor shapes or wrong data types, can lead to errors showing up in the TensorFlow Lite interpreter. Making sure the input matched the model's expected shape and type required careful attention. Another challenge I faced was understanding how a full TensorFlow model differs from a TensorFlow Lite model, and how each one is used in different settings.

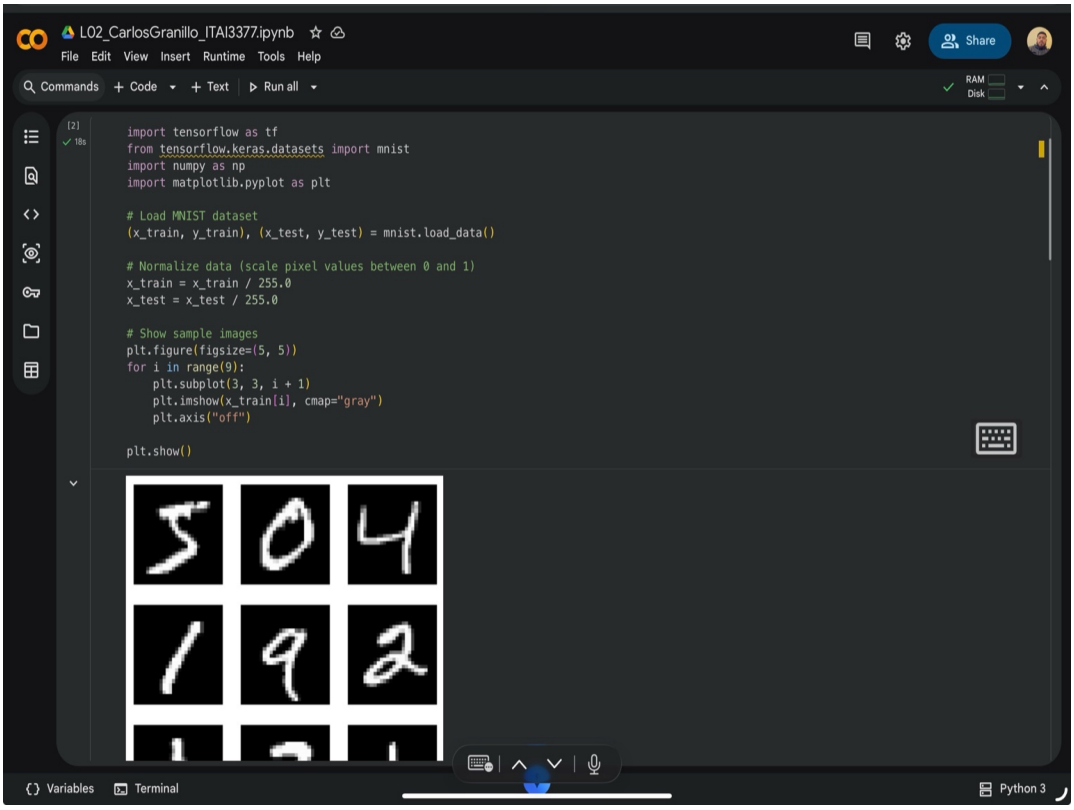
After completing this lab, I can say that I learned more about how neural networks can recognize patterns in images and how important it is to preprocess data correctly. I also learned how to convert a trained model into TensorFlow Lite and run predictions using an interpreter. TensorFlow Lite is useful for real-world applications because it allows models to run directly on devices like smartphones, cameras, or IoT systems as well. This means predictions can be made without sending data to the cloud, which is faster, uses less power, and most importantly protects your privacy. Overall, this assignment helped me understand the full path from training a machine learning model to deploying it efficiently, and how TensorFlow Lite makes AI practical on everyday devices.

Jupyter Notebook Content

```
!python --version !pip show tensorflow

import tensorflow as tf from tensorflow.keras.datasets import mnist import numpy as
np import matplotlib.pyplot as plt # Load MNIST dataset (x_train, y_train), (x_test,
y_test) = mnist.load_data() # Normalize data (scale pixel values between 0 and 1)
x_train = x_train / 255.0 x_test = x_test / 255.0 # Show sample images
plt.figure(figsize=(5, 5)) for i in range(9): plt.subplot(3, 3, i + 1)
plt.imshow(x_train[i], cmap="gray") plt.axis("off") plt.show()

# Define model architecture model = tf.keras.models.Sequential([
tf.keras.layers.Flatten(input_shape=(28, 28)), # Input layer
tf.keras.layers.Dense(128, activation='relu'), # Hidden layer
tf.keras.layers.Dense(10, activation='softmax') # Output layer (10 classes) ]) #
Compile model model.compile( optimizer='adam',
loss='sparse_categorical_crossentropy', metrics=['accuracy'] ) # Train model
model.fit( x_train, y_train, epochs=5, validation_data=(x_test, y_test) ) # Save
trained model model.save("mnist_model.h5") print("Model training complete and saved
as mnist_model.h5")

# Load trained model model = tf.keras.models.load_model("mnist_model.h5") # Convert
to TensorFlow Lite converter = tf.lite.TFLiteConverter.from_keras_model(model)
tflite_model = converter.convert() # Save the converted model with
open("mnist_model.tflite", "wb") as f: f.write(tflite_model) print("Model
successfully converted and saved as mnist_model.tflite")

# Load TensorFlow Lite model interpreter =
tf.lite.Interpreter(model_path="mnist_model.tflite") interpreter.allocate_tensors()
# Get input and output tensor details input_details =
interpreter.get_input_details() output_details = interpreter.get_output_details()
print("Input Details:", input_details) print("Output Details:", output_details)

# Select a test image test_image = x_test[0].astype(np.float32) # Match model input
type test_image = np.expand_dims(test_image, axis=0) # Add batch dimension # Set the
input tensor interpreter.set_tensor(input_details[0]['index'], test_image) # Run
inference interpreter.invoke() # Get the prediction output_data =
interpreter.get_tensor(output_details[0]['index']) predicted_label =
np.argmax(output_data) # Display the image and prediction plt.imshow(x_test[0],
cmap="gray") plt.title(f"Predicted Label: {predicted_label}, Actual Label:
{y_test[0]}") plt.axis("off") plt.show()
```

Colab Notebook Link

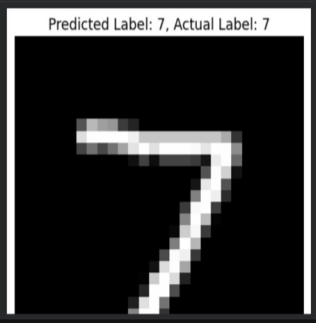
https://colab.research.google.com/drive/1RYFBqAZ0ZG8MiDU9zSZJ_2F2pJa5vNAI?authuser=0#scrollTo=QubkJLoG-WD0



Commands + Code + Text Run all

[6]

```python
# Select a test image
test_image = x_test[0].astype(np.float32)  # Match model input type
test_image = np.expand_dims(test_image, axis=0)  # Add batch dimension

# Set the input tensor
interpreter.set_tensor(input_details[0]['index'], test_image)

# Run inference
interpreter.invoke()

# Get the prediction
output_data = interpreter.get_tensor(output_details[0]['index'])
predicted_label = np.argmax(output_data)

# Display the image and prediction
plt.imshow(x_test[0], cmap="gray")
plt.title(f"Predicted Label: {predicted_label}, Actual Label: {y_test[0]}")
plt.axis("off")
plt.show()
```



Variables Terminal Python 3